Lucrarea 4 – Reducerea paralelă. Sortarea. Recursivitate

Laborator Algoritmi Paraleli și Distribuiți – rezolvarea exercițiilor · File-Mitita Traian, 441A

Toate versiunile sunt în același program (`L4/Program.cs`), ca în fișa de laborator. Vectorul are elementele egale cu 1, deci suma corectă este N. Am rulat cu **N = 128, P = 16** (valorile din fișă) și cu **N = 1024, P = 16**, pe macOS 26.7.2, Apple M2, 8 procesoare logice, .NET 8 Release, 13.11.2026 17:47. N și P se pot da la rulare: `L4.exe <N> <P>`.

Două corecturi față de codul din fișă: fiecare versiune are propriul `Stopwatch` (în fișă, la V1 se pornea `timerSeq` și se afișa `timerPar1`), iar înainte de fiecare versiune vectorul se reinițializează cu `Init()`, pentru că V3 și V5 scriu sumele parțiale chiar în vectorul `a`.

EXERCIȚIUL 1 – Versiunea paralelă V1

N threaduri, fiecare adună elementul său la variabila partajată `sTemp`, fără sincronizare:

```
public static double ReductionParV1()
{
    sTemp = 0;

    Thread[] childThreads = new Thread[N];
    int i;
    for (i = 0; i < N; i++)
    {
        childThreads[i] = new Thread(ThreadFunctionV1);
        childThreads[i].Start(i);
    }
    for (i = 0; i < N; i++)
    {
        childThreads[i].Join();
    }

    return sTemp;
}

public static void ThreadFunctionV1(object parameter)
{
    int idx = (int)parameter;
    sTemp += a[idx];
}
```

```
Sum(A)=128; Seq time = 0,048 ms
Sum(A)=128; ParV1 time = 5,368 ms
Sum(A)=128; ParV1 time = 5,3665 ms
Sum(A)=128; ParV1 time = 4,9078 ms
Sum(A)=128; ParV1 time = 5,271 ms
Sum(A)=128; ParV1 time = 5,0561 ms
Sum(A)=128; ParV1 time = 4,5968 ms
Sum(A)=128; ParV1 time = 4,7866 ms
Sum(A)=128; ParV1 time = 4,81 ms
Sum(A)=128; ParV1 time = 5,3839 ms
Sum(A)=128; ParV1 time = 4,4845 ms
V1: 0 rezultate gresite din 10
```

- **Timp:** în medie 5,00 ms la N = 128 și 48,05 ms la N = 1024, față de 0,05 ms serial. Aproape tot timpul înseamnă crearea și pornirea a N threaduri, pentru o singură adunare fiecare.
- **Corectitudine:** în cele 10 rulări suma a ieșit corectă de fiecare dată (și la N = 1024). Asta nu înseamnă că V1 e corectă: `sTemp += a[idx]` înseamnă citire, adunare și scriere, nu o operație atomică. Dacă două threaduri citesc aceeași valoare a lui `sTemp`, una dintre adunări se pierde. Pe calculatorul meu fiecare thread termină adunarea înainte să pornească următorul (pornirea

unui thread durează mult mai mult decât o adunare), de aceea eroarea nu a apărut, dar rezultatul nu e garantat.

EXERCIȚIUL 2 – V1 cu mutex

Accesul la `sTemp` se protejează cu un `Mutex`, deci un singur thread modifică suma la un moment dat:

```
public static void ThreadFunctionV1Mutex(object parameter)
{
    int idx = (int)parameter;
    mutex.WaitOne();
    sTemp += a[idx];
    mutex.ReleaseMutex();
}
```

```
Sum(A)=128; ParV1 + mutex time = 4,9437 ms
```

Rezultatul este corect garantat. Timpul (4,94 ms la N = 128, 39,50 ms la N = 1024) este practic cel al lui V1: accesul la `sTemp` este serializat, deci execuția este de fapt serială, plus costul threadurilor și al mutex-ului.

EXERCIȚIUL 3 – Versiunea paralelă V2

Cele N elemente se împart în P = 16 partiții de N/P elemente; fiecare thread calculează serial suma partiției sale în `pSum[idx]`, iar threadul principal adună sumele parțiale după `Join`:

```
public static double ReductionParV2()
{
    sTemp = 0;
    pSum = new double[P];
    Thread[] childThreads = new Thread[P];
    int i;
    for (i = 0; i < P; i++)
    {
        childThreads[i] = new Thread(ThreadFunctionV2);
        childThreads[i].Start(i);
    }
    for (i = 0; i < P; i++)
    {
        childThreads[i].Join();
        sTemp += pSum[i];
    }

    return sTemp;
}

public static void ThreadFunctionV2(object parameter)
{
    int idx = (int)parameter;
    pSum[idx] = 0;
    for (int i = (N / P) * idx; i < (N / P) * (idx + 1); i++)
    {
        pSum[idx] += a[i];
    }
}
```

```
Sum(A)=128; ParV2 time = 0,823 ms
```

Rezultatul este corect (fiecare thread scrie doar în `pSum[idx]`, nu există variabilă modificată de mai multe threaduri). Timp: 0,82 ms la N = 128 și 0,82 ms la N = 1024, mult mai puțin decât V1, pentru

că se creează doar 16 threaduri. Pentru vectori atât de mici rămâne totuși mai lent decât varianta serială: costul pornirii threadurilor e mai mare decât adunările.

EXERCIȚIUL 4 – Versiunea paralelă V3 (barieră)

N/2 threaduri, $\log_2 N$ pași; la fiecare pas toate threadurile se sincronizează la barieră, apoi threadurile cu `idx % (2·i) == 0` adună elementul aflat la distanța i:

```
public static double ReductionParV3()
{
    sTemp = 0;
    Thread[] childThreads = new Thread[N / 2];
    barier = new Barrier(N / 2);
    int i;
    for (i = 0; i < N / 2; i++)
    {
        childThreads[i] = new Thread(ThreadFunctionV3);
        childThreads[i].Start(2 * i);
    }
    for (i = 0; i < N / 2; i++)
    {
        childThreads[i].Join();
    }
    sTemp = a[0];
    return sTemp;
}

public static void ThreadFunctionV3(object parameter)
{
    int idx = (int)parameter;
    for (int i = 1; i <= N / 2; i *= 2)
    {
        barier.SignalAndWait();
        if (idx % (2 * i) == 0)
            a[idx] += a[idx + i];
    }
}
```

```
Sum(A)=128; ParV3 time = 7,8061 ms
```

Rezultatul este corect (suma totală ajunge în `a[0]`). Timpul este cel mai mare: 7,81 ms la N = 128 și 131,35 ms la N = 1024. Deși complexitatea teoretică e O(log N), pe un CPU cu 8 procesoare cele N/2 threaduri nu pot lucra toate în același timp, iar fiecare pas așteaptă la barieră toate threadurile. Cu N = 8192 (4096 de threaduri) programul nu a mai terminat.

EXERCIȚIUL 5 – Versiunea paralelă V4 = V2 + V3

Fiecare din cele P threaduri calculează suma partiției sale (ca la V2), apoi sumele parțiale din `pSum` se reduc în $\log_2 P$ pași cu barieră (ca la V3), de aceea P trebuie să fie putere a lui 2:

```
public static double ReductionParV4()
{
    sTemp = 0;
    pSum = new double[P];
    Thread[] childThreads = new Thread[P];
    barier = new Barrier(P);
    int i;
    for (i = 0; i < P; i++)
    {
        childThreads[i] = new Thread(ThreadFunctionV4);
        childThreads[i].Start(i);
    }
    for (i = 0; i < P; i++)
    {
        childThreads[i].Join();
    }
    sTemp = pSum[0];
    return sTemp;
}

public static void ThreadFunctionV4(object parameter)
{
    int idx = (int)parameter;
    pSum[idx] = 0;
    for (int i = (N / P) * idx; i < (N / P) * (idx + 1); i++)
    {
        pSum[idx] += a[i];
    }
    for (int i = 1; i <= P / 2; i *= 2)
    {
        barier.SignalAndWait();
        if (idx % (2 * i) == 0)
            pSum[idx] += pSum[idx + i];
    }
}
```

```
Sum(A)=128; ParV4 time = 0,9705 ms
```

Rezultatul este corect. Timp: 0,97 ms la N = 128 și 1,18 ms la N = 1024, apropiat de V2: cu P = 16 partea finală are doar $\log_2 16 = 4$ pași în loc de 16 adunări, diferență mică față de costul pornirii threadurilor și al barierei.

EXERCIȚIUL 6 – Versiunea paralelă V5 (recursivă)

Fiecare thread își împarte partiția în două și pornește câte un thread pentru fiecare jumătate, până la partiții de 2 elemente; după `Join` pe cei doi copii adună cele două sume parțiale. Nu mai e nevoie de barieră, pentru că părintele așteaptă copiii:

```
public static double ReductionParV5()
{
    sTemp = 0;
    Thread[] childThreads = new Thread[2];
    int i;
    int j = 0;
    for (i = 0; i < N; i += N / 2, j++)
    {
        childThreads[j] = new Thread(ThreadFunctionV5);
        childThreads[j].Start(new ThreadParam { pos = i, len = N / 2 });
    }
    for (i = 0; i < 2; i++)
    {
        childThreads[i].Join();
    }
    sTemp = a[0] + a[N / 2];
    return sTemp;
}

public static void ThreadFunctionV5(object parameter)
{
    var p = (ThreadParam)parameter;
    if (p.len > 2)
    {
        Thread[] childThreads = new Thread[2];
        int i;
        int j = 0;
        for (i = 0; i < p.len; i += p.len / 2, j++)
        {
            childThreads[j] = new Thread(ThreadFunctionV5);
            // pozitia copilului este relativa la partitia parintelui: p.pos + i
            childThreads[j].Start(new ThreadParam { pos = p.pos + i, len = p.len / 2 });
        }
        for (i = 0; i < 2; i++)
        {
            childThreads[i].Join();
        }
        a[p.pos] += a[p.pos + p.len / 2];
    }
    else
    {
        a[p.pos] += a[p.pos + 1];
    }
}
```

În codul din fișă copiii primeau `pos = i`, adică poziția față de începutul vectorului, nu față de partiția părintelui. Pentru jumătatea a doua a vectorului, copiii ar fi lucrat tot pe începutul vectorului, iar suma ar fi ieșit greșită (urmărind pașii pe hârtie, la N = 128 ar ieși 407 în loc de 128), așa că am folosit `pos = p.pos + i`.

```
Sum(A)=128; ParV5 time = 4,3152 ms
```

Rezultatul este corect. Timp: 4,32 ms la N = 128 și 36,13 ms la N = 1024: se creează în total aproape N threaduri, la fel ca la V1, deci și timpul e de același ordin.

EXERCIȚIUL 7 – Minim, maxim și medie

Pentru acest exercițiu vectorul are valori aleatoare între 0 și 999. Varianta serială parcurge vectorul o dată pentru minim, o dată pentru maxim și o dată pentru sumă; media = suma / N. Varianta paralelă folosește ideea de la V2: fiecare din cele P threaduri calculează minimul, maximul și suma partiției sale, iar threadul principal le combină după `Join`:

```
public static double MinSeq()
{
    double m = a[0];
    for (int i = 1; i < N; i++)
    {
        if (a[i] < m) m = a[i];
    }
    return m;
}

public static double MaxSeq()
{
    double m = a[0];
    for (int i = 1; i < N; i++)
    {
        if (a[i] > m) m = a[i];
    }
    return m;
}
```

```
public static void MinMaxMediePar(out double min, out double max, out double medie)
{
    pMin = new double[P];
    pMax = new double[P];
    pSum = new double[P];
    Thread[] childThreads = new Thread[P];
    int i;
    for (i = 0; i < P; i++)
    {
        childThreads[i] = new Thread(ThreadFunctionMinMax);
        childThreads[i].Start(i);
    }
    min = double.MaxValue;
    max = double.MinValue;
    double suma = 0;
    for (i = 0; i < P; i++)
    {
        childThreads[i].Join();
        if (pMin[i] < min) min = pMin[i];
        if (pMax[i] > max) max = pMax[i];
        suma += pSum[i];
    }
    medie = suma / N;
}

public static void ThreadFunctionMinMax(object parameter)
{
    int idx = (int)parameter;
    int start = (N / P) * idx;
    pMin[idx] = a[start];
    pMax[idx] = a[start];
    pSum[idx] = 0;
    for (int i = start; i < (N / P) * (idx + 1); i++)
    {
        if (a[i] < pMin[idx]) pMin[idx] = a[i];
        if (a[i] > pMax[idx]) pMax[idx] = a[i];
        pSum[idx] += a[i];
    }
}
```

```
B: 815 990 563 411 11 54 698 448
Seq:  min=8; max=995; media=499,0078125; time = 0,1096 ms
Par:  min=8; max=995; media=499,0078125; time = 0,7425 ms
```

```
Seq:  min=0; max=999; media=497,8623046875; time = 0,1163 ms
Par:  min=0; max=999; media=497,8623046875; time = 0,9079 ms
```

Ambele variante dau același minim, maxim și aceeași medie. Și aici varianta paralelă e mai lentă pentru N mic, din cauza pornirii celor 16 threaduri.

Comparația timpilor (ms)

Versiune	Complexitate	N = 128	N = 1024
Serial	O(N)	0,05	0,05
V1 (fără sincronizare, medie 10 rulări)	–	5,00	48,05
V1 + mutex	O(N), acces serializat	4,94	39,50
V2 (P partiții)	O(N/P) + O(P)	0,82	0,82
V3 (barieră)	O(log N)	7,81	131,35
V4 = V2 + V3	O(N/P) + O(log P)	0,97	1,18
V5 (recursiv)	O(log N)	4,32	36,13

Toate versiunile sincronizate dau suma corectă. Pentru vectori mici, varianta serială e cea mai rapidă. Dintre cele paralele, V2 și V4 sunt cele mai rapide, pentru că folosesc puține threaduri. V1, V3 și V5 creează un număr de threaduri proporțional cu N, deci timpul lor crește cu N, chiar dacă V3 are teoretic complexitatea cea mai mică.